HOW TO: Network Services Setup (Routing, DNS, DHCP, FTP, NFS)

Overview

This guide walks you through setting up and testing **five key network services** on your virtual machines:

1. **Routing and NAT Configuration**
2. **DNS Server Setup**
3. **DHCP Server Setup**
4. **FTP Server Configuration**
5. **NFS File Sharing**

Each section builds on the previous one.

Network Settings Used in This Guide

To make the setup easy to follow, this guide uses the following example network:

- **Server IP:** `192.168.56.10`
- **Client (DHCP Range):** `192.168.56.150 – 192.168.56.200`
- **Subnet:** `192.168.56.0/24`
- **Domain Name:** `student22503.lan`
- **Router / DNS / Gateway:** `192.168.56.10`

These values are used in every configuration (Routing, DNS, DHCP, FTP, NFS) so each service works together smoothly.

Customising for Your Own Network

You can **replace any of the values above** with your own:

- Change the **server IP** to another static address
- Use a different **domain name**

- Adjust the **DHCP IP range**
- Modify network interfaces to match your VM setup

As long as you update the same values consistently across all configuration files, everything will work perfectly.

This guide simply gives you a clean, structured template --- you can personalise it to match your environment.

Routing and NAT Configuration

In this section, you will learn how to turn your Linux machine into a working router. By the end, your server will forward traffic, share the internet connection, and allow a client VM to reach external websites. You will configure interfaces, enable IP forwarding, set up NAT, create firewall rules, and fully test client → router → internet communication.

1. Configuring a Second Network Interface

VirtualBox Setup

1. Power off your VM.
2. Go to **Settings → Network → Adapter 2**.
3. Enable **Host-only Adapter**.
4. Ensure **Cable Connected** is checked.
5. Boot the VM.

Interface Configuration

Check for the new interface:

```
# Shows all network interfaces on the system. Helps you confirm that enp0s8
exists.
ip link show
```

Create a new Netplan configuration:

```
# Opens (or creates) a Netplan file so you can configure the second network
interface.
sudo nano /etc/netplan/second-interface.yaml
```

Add:

```
# Netplan config that gives the new interface (enp0s8)
# a static IP address on the Host-Only network.
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s8:
      addresses:
        - 192.168.56.10/24
```

Apply changes:

```
# Applies the new network settings.
sudo netplan apply
# Shows the IP address and status of the new interface.
ip addr show enp0s8
```

---

2. Enabling IP Forwarding and Routing

Enable forwarding temporarily:

```
# Turns on IP forwarding temporarily (until reboot).
# This allows the VM to act as a router.
sudo sysctl -w net.ipv4.ip_forward=1
```

Make it permanent:

```
# Opens the config file where permanent system settings are stored.
sudo nano /etc/sysctl.conf
```

Uncomment or add:

```
# Makes IP forwarding permanent, so it survives reboots.
net.ipv4.ip_forward=1
```

---

3. Configuring NAT with iptables

Enable NAT and forwarding:

```
# NAT rule: hides internal IPs behind the router’s external IP (basic
internet sharing).
sudo iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE

# Allows traffic to go from internal network to external network.
sudo iptables -A FORWARD -i enp0s8 -o enp0s3 -j ACCEPT

# Allows return traffic from the internet back to the client.
sudo iptables -A FORWARD -i enp0s3 -o enp0s8 -m state --state
ESTABLISHED,RELATED -j ACCEPT
```

Verify rules:

```
# Shows NAT table rules to confirm MASQUERADE is set.
sudo iptables -t nat -L -n -v

# Shows forwarding rules to ensure routing is allowed.
sudo iptables -L FORWARD -n -v
```

---

4. Implementing Firewall Security

Create a script for consistency:

```
# Creates a firewall script so your rules are consistent every time.
sudo nano /root/firewall.sh
```

Add:

```
#!/bin/bash
# Firewall configuration for router

# Removes all current firewall rules (filter table).
iptables -F
# Clears NAT rules.
iptables -t nat -F
# Deletes any custom chains.
iptables -X
```

```
# Drops all incoming traffic by default.
iptables -P INPUT DROP
# Blocks all forwarded traffic unless allowed.
iptables -P FORWARD DROP
# Allows all outgoing connections.
iptables -P OUTPUT ACCEPT

# Allows the loopback interface (local machine traffic).
iptables -A INPUT -i lo -j ACCEPT

# Allows responses to traffic you already started (e.g., web browsing).
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
# Same as above, but for forwarded packets (router traffic).
iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

# Allows SSH connections from the internal network.
iptables -A INPUT -p tcp --dport 22 -i enp0s8 -j ACCEPT

# Allows DNS requests.
iptables -A INPUT -p udp --dport 53 -j ACCEPT
iptables -A INPUT -p tcp --dport 53 -j ACCEPT

# Allows DHCP traffic.
iptables -A INPUT -p udp --dport 67:68 -j ACCEPT

# Allow ping from internal network
iptables -A INPUT -p icmp --icmp-type echo-request -i enp0s8 -j ACCEPT

# Allows internal clients to reach the internet.
iptables -A FORWARD -i enp0s8 -o enp0s3 -j ACCEPT

# Enables NAT (internet sharing) on the external interface.
iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE

echo "Firewall configured successfully"
```

Make executable and run:

```
# Makes the script executable.
sudo chmod +x /root/firewall.sh

# Runs your firewall rules.
sudo /root/firewall.sh
```

Check:

```
# Checks filter rules.
sudo iptables -L -n -v

# Tests if the router can reach the internet.
ping -c 2 8.8.8.8
```

---

5. Make Configuration Persistent

Install persistence tools:

```
# Installs tools so firewall rules are saved after reboot.
sudo apt install iptables-persistent netfilter-persistent

# Saves current iptables rules permanently.
sudo netfilter-persistent save
```

Create a systemd service:

```
# Creates a service that runs your firewall script at boot.
sudo nano /etc/systemd/system/firewall.service
```

Add:

```
[Unit]
Description=Configure Firewall
After=network.target

[Service]
Type=oneshot
ExecStart=/root/firewall.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
# Makes the service start automatically on boot.
sudo systemctl enable firewall.service

# Starts the service right now.
sudo systemctl start firewall.service
```

---

6. Testing Router Configuration

Clone your VM to create a **Client-VM**:

1. Shutdown your router VM.
2. In VirtualBox → Right-click → **Clone** → “Full Clone”.
3. Disable **Adapter 1**, keep **Adapter 2** (Host-only).
4. Start the Client VM.

Client Configuration

```
# Create a Netplan file for the client VM's network settings.
sudo nano /etc/netplan/client.yaml
```

Add:

```
# Using Netplan version 2. This keeps settings modern and compatible.
version: 2

# Using networkd because it's lightweight and perfect for servers.
renderer: networkd

ethernets:
  enp0s8:
    # Static IP chosen so the client always has a predictable address.
    # This makes testing, routing, and troubleshooting much easier.
    addresses:
      - 192.168.56.20/24 # /24 keeps the network simple and supports up to
254 devices.

    routes:
      - to: default # Setting a default route makes all traffic go to the
server/router.
        via: 192.168.56.10 # Server acts as the gateway for internet access
and internal routing.
```

```
    nameservers:
      # External DNS chosen (Google). Reliable, fast, and avoids depending
on the local DNS.
      addresses: [8.8.8.8, 8.8.4.4]
```

Apply and test:

```
# Activates the client's network settings.
sudo netplan apply

# Checks if the client can reach the router.
ping -c 2 192.168.56.10

# Checks if the client can reach the internet through the router.
ping -c 2 8.8.8.8
```

If both pings succeed, routing and NAT are functioning correctly.

---

7. Verification Checklist

Test	Expected Result
`sysctl net.ipv4.ip_forward`	1 (enabled)
`iptables -t nat -L -n -v`	MASQUERADE rule visible
Internal → external ping	Works
Firewall script active	Rules applied
`ping 8.8.8.8` from client	Success
`/root/firewall.sh` executes without error	No Error Messages

Routing and NAT is now fully set up

You now have a fully functioning router that can forward packets, translate addresses (NAT), and securely control traffic through firewall rules. Your client can reach the router and the internet, and your routing setup is stable and persistent across reboots. This creates the foundation for all other services in your network.

Routing & NAT – Troubleshooting Common Issues

This section gives troubleshooting steps for common Routing and NAT issues based on your configuration.

---

1. Check That the Network Interface Is Configured

```
# Shows all interfaces. Confirm enp0s8 has 192.168.56.10/24
ip a
```

If **enp0s8** shows *DOWN*:

```
# Brings interface up
sudo ip link set enp0s8 up

# Re-applies network settings
sudo netplan apply
```

---

2. Validate IP Forwarding

```
# Should return 1
sysctl net.ipv4.ip_forward
```

If it returns 0:

```
# Temporary enable
sudo sysctl -w net.ipv4.ip_forward=1

# Make permanent
sudo nano /etc/sysctl.conf
# Ensure: net.ipv4.ip_forward=1
```

---

3. Check the NAT (MASQUERADE) Rule

```
# Confirms POSTROUTING MASQUERADE exists
sudo iptables -t nat -L -n -v
```

If MASQUERADE is missing:

```
# Re-add NAT rule
sudo iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE
```

---

4. Check Forwarding Rules

```
# Shows forwarding rules
sudo iptables -L FORWARD -n -v
```

If traffic is being **dropped**:

```
# Re-add rules that allow forwarding
sudo iptables -A FORWARD -i enp0s8 -o enp0s3 -j ACCEPT
sudo iptables -A FORWARD -i enp0s3 -o enp0s8 -m state --state
ESTABLISHED,RELATED -j ACCEPT
```

---

5. Test Router → Internet Connectivity

```
# Ensures the router itself can reach the internet
ping -c 2 8.8.8.8
```

If it fails:

- Check that **Adapter 1** is NAT in VirtualBox
- Check your host's internet connection
- Restart networking:

```
sudo systemctl restart systemd-networkd
```

---

6. Test Client → Router

On the client VM:

```
 # Tests if routing VM is reachable
ping -c 2 192.168.56.10
```

If it fails:

- Check client IP configuration
- Make sure client interface enp0s8 is UP

---

7. Test Client → Internet Through NAT

```
# Should work through NAT router
ping -c 2 8.8.8.8
```

If it does **NOT** work but router can ping:

- NAT rule missing
- Forwarding rules missing
- Firewall script not active

---

8. Check Firewall Script Status

```
# Should be active (exited)
sudo systemctl status firewall.service
```

If it is **not active**:

```
# Start it
sudo systemctl start firewall.service

# Enable on boot
sudo systemctl enable firewall.service
```

---

9. Persistence Problems After Reboot

If rules disappear after reboot:

```
# Saves rules
sudo netfilter-persistent save

# Reloads saved rules
sudo netfilter-persistent reload
```

---

10. Quick Reset (If Everything Breaks)

```
sudo iptables -F
sudo iptables -t nat -F
sudo iptables -X

# Re-run your known-good script
sudo /root/firewall.sh
```

DNS Server Configuration

This section teaches you how to build a complete DNS server using Bind9. When finished, your server will resolve both internal names (like server.student22503.lan) and external domains (like google.com). You will set up forward and reverse zones, use DNS forwarders, validate zone files, and test name resolution from both server and client.

---

1. Install and Enable BIND9

```
# Updates the package list so your system knows the latest versions.
sudo apt update

# Installs the BIND9 DNS server + tools for checking zone files +
documentation.
sudo apt install bind9 bind9-utils bind9-doc -y

# Makes BIND9 start automatically every time the system boots.
sudo systemctl enable named

# Starts the DNS server right now.
```

```
sudo systemctl start named

# Shows whether the DNS service is running. It should say "active
(running)".
sudo systemctl status named
```

Check that it shows `active (running)` before continuing.

---

2. Configure Global DNS Options

Edit the file:

```
# Opens the main DNS options file where you set recursion, forwarders, etc.
sudo nano /etc/bind/named.conf.options
```

Add or update it to match:

```
options {
    directory "/var/cache/bind";
    // Where BIND stores its working files (cache, etc.)

    allow-query { any; };
    // Allow anyone on the network to ask this DNS server questions.
    // Simple for a simple setup, but not safe for public internet.

    recursion yes;
    // Turn on recursion so this server can look up external websites
    // (e.g., google.com) on behalf of clients.

    forwarders {
        8.8.8.8;   // Google DNS — fast and reliable
        1.1.1.1;   // Cloudflare DNS — privacy-focused
    };
    // These DNS servers are used when your own server does not know an
answer.

    dnssec-validation no;
    // Turn off DNSSEC checks. Makes simple setups easier.
    // Not recommended for real production servers.

    listen-on { any; };
    // DNS server listens on all IPv4 interfaces.
```

```
    listen-on-v6 { any; };
    // DNS server listens on all IPv6 interfaces.
};
```

Save and exit (`Ctrl + O`, `Ctrl + X`).

---

3. Define Forward and Reverse Zones

Edit:

```
# Local DNS zone configuration.
# This is where you declare your custom domain and reverse lookup zone.
sudo nano /etc/bind/named.conf.local
```

Replace the contents with:

```
//
// Local zone configuration
//

// Forward zone for your custom domain
zone "student22503.lan" {
    type master;
    // This DNS server is the primary (master) for this domain.

    file "/var/cache/bind/db.student22503.lan";
    // This is the zone file that holds all the DNS records for the domain.
};

// Reverse zone for your subnet
zone "56.168.192.in-addr.arpa" {
    type master;
    // This server also controls the reverse lookup zone
    // (turning IP addresses back into names).

    file "/var/cache/bind/db.192.168.56";
    // Zone file for PTR records for 192.168.56.x addresses.
};
```

---

4. Create the Forward Zone File

```
# Creates the forward zone database.
# This file contains A records, CNAMEs, and the SOA record.
sudo nano /var/cache/bind/db.student22503.lan
```

Paste in:

```
$TTL 3600                    ; Default time-to-live for all records (1
hour)

$ORIGIN student22503.lan.     ; Sets the base domain for all records below

@ IN SOA server.student22503.lan. admin.student22503.lan. (
    2025110901 ; Serial number — increase this when you update the file
    3600       ; Refresh — how often secondary servers check for updates
    1800       ; Retry — if refresh fails, retry time
    604800     ; Expire — how long secondaries keep data without updates
    900        ; Negative Cache TTL — how long “not found” answers are
cached
)

@       IN  NS     server.student22503.lan.   ; This domain’s authoritative
DNS server
@       IN  A      192.168.56.10              ; A record for the root of the
domain
server  IN  A      192.168.56.10              ; Server hostname → server IP
client  IN  A      192.168.56.20              ; Client hostname → client IP
www     IN  CNAME  server.student22503.lan.   ; www alias pointing to the
server
```

Save and close.

---

5. Create the Reverse Zone File

```
# Reverse DNS zone database.
# Maps the last octet of an IP to hostname.
sudo nano /var/cache/bind/db.192.168.56
```

Add this:

```
$TTL 3600                               ; Default TTL for all records (1
hour)

$ORIGIN 56.168.192.in-addr.arpa.        ; Base domain for reverse DNS
lookups

@ IN SOA server.student22503.lan. admin.student22503.lan. (
    2025110901 ; Serial number — increase when making changes
    3600       ; Refresh — how often secondaries check for updates
    1800       ; Retry — wait before retrying after a failed refresh
    604800     ; Expire — how long secondaries keep data without updates
    900        ; Negative Cache TTL — cache time for “not found” responses
)

@  IN  NS  server.student22503.lan.     ; This zone’s authoritative DNS
server

10 IN  PTR server.student22503.lan.     ; 192.168.56.10 → server
20 IN  PTR client.student22503.lan.     ; 192.168.56.20 → client
```

Save and close.

---

6. Validate and Restart BIND9

```
# Checks that BIND config files are valid.
sudo named-checkconf

# Verifies forward zone syntax.
sudo named-checkzone student22503.lan /var/cache/bind/db.student22503.lan

# Verifies reverse zone syntax.
sudo named-checkzone 56.168.192.in-addr.arpa /var/cache/bind/db.192.168.56

# Restarts BIND9 so changes apply.
sudo systemctl restart named

# Shows if DNS server is running correctly.
sudo systemctl status named
```

All checks should say "OK". The status should show `active (running)`.

---

7. Test from the Server

```
# Test domain to IP resolution.
dig @localhost student22503.lan

# Should return 192.168.56.10
dig @localhost server.student22503.lan

# Should return 192.168.56.20
dig @localhost client.student22503.lan

# Reverse lookup to hostname.
dig @localhost -x 192.168.56.10
dig @localhost -x 192.168.56.20

# Tests external domain resolution through forwarders.
dig @localhost google.com
```

Expected results:

- Local names return your IPs (192.168.56.10 / 192.168.56.20)
- Reverse lookups return correct hostnames
- `google.com` returns multiple IPs

---

8. Test from the Client (192.168.56.20)

Edit the client's resolver file temporarily:

```
# Temporarily sets the client VM to use your DNS server.
sudo nano /etc/resolv.conf
```

Replace contents with:

```
nameserver 192.168.56.10
search student22503.lan
```

Then test:

```
# Forward lookup from the client.
dig @192.168.56.10 server.student22503.lan
dig @192.168.56.10 client.student22503.lan
```

```
# Reverse lookup from the client.
dig @192.168.56.10 -x 192.168.56.10

# Checks if recursive DNS works from the client.
dig @192.168.56.10 google.com
```

All should return valid results.

---

9. Verification Summary

Test	Expected Result
`dig server.student22503.lan`	Returns 192.168.56.10
`dig client.student22503.lan`	Returns 192.168.56.20
`dig -x 192.168.56.10`	Returns server.student22503.lan
`dig google.com`	Returns public IPs
`systemctl status named`	Shows "active (running)"

DNS is now fully set up

Your DNS server is now fully operational, resolving internal domain names and forwarding external queries correctly. Forward and reverse lookups work, the client uses your DNS server automatically, and the configuration is stable with no errors. This DNS setup supports all future services in your network.

DNS Troubleshooting Common Issues

This section gives troubleshooting steps for common BIND9 DNS Server issues based on your configuration.

---

1. Check if BIND9 Is Running

```
# Should show: active (running)
systemctl status named
```

If it is **not running**:

```
# Restart DNS service
sudo systemctl restart named

# Shows the error that stopped BIND9
sudo journalctl -xe
```

---

2. Validate DNS Config Files

```
# Checks main config syntax
sudo named-checkconf
```

If there is an error, it will point to the exact file + line.

---

3. Validate Zone Files

```
# Forward zone check
sudo named-checkzone student22503.lan /var/cache/bind/db.student22503.lan

# Reverse zone check
sudo named-checkzone 56.168.192.in-addr.arpa /var/cache/bind/db.192.168.56
```

If it shows errors like *"unknown RR type"* or *"out of zone data"* → fix the line shown.

---

4. Check That DNS Is Listening on the Correct IP

```
# Shows if named is listening on port 53 (UDP/TCP)
sudo ss -tulpn | grep named
```

If nothing appears:

- Check `listen-on { any; };` is set
- Restart the service

---

5. Test Local DNS Resolution (Server)

```
# Should return 192.168.56.10
dig @localhost server.student22503.lan

# Should return 192.168.56.20
dig @localhost client.student22503.lan
```

If it fails:

- Zone files may be missing an A record
- Zone path in `named.conf.local` may be wrong

---

6. Test Reverse DNS

```
# Should return server.student22503.lan
dig @localhost -x 192.168.56.10
```

If this fails:

- PTR line may be wrong (example: “10 IN PTR…”)

---

7. Check Forwarders / Internet DNS

```
# Tests recursion + forwarders
dig @localhost google.com
```

If it hangs or fails:

- Your server may not have internet
- Forwarders might be blocked
- Recursion may be disabled

8. Client Cannot Resolve DNS

Step 1 — Check DNS Server IP

```
# Should show nameserver 192.168.56.10
cat /etc/resolv.conf
```

Step 2 — Ping Test

```
# Tests if the client reaches the server
ping -c 2 192.168.56.10
```

Step 3 — DNS Query From Client

```
dig @192.168.56.10 server.student22503.lan
```

If client cannot query the DNS server:

- Firewall may block UDP/TCP 53
- Client may have wrong gateway or DNS address

---

9. Restart Everything Cleanly

```
# Restart DNS server
sudo systemctl restart named

# Reload zone files safely
sudo rndc reload
```

---

10. Quick Reset (If Everything Breaks)

```
# Deletes journal files that cause zone lock errors
sudo rm -rf /var/cache/bind/*.jnl
sudo systemctl restart named
```

DHCP Server Configuration

In this section, you set up a DHCP server that automatically gives out IP addresses and network settings to clients. By the end, your client VM will receive an IP, gateway, DNS, subnet mask, and domain name without any manual configuration. This makes devices on your network easy to manage and consistent.

---

1. Install and Enable ISC DHCP Server

```
# Updates the package list to get the latest versions.
sudo apt update

# Installs the ISC DHCP server software.
sudo apt install isc-dhcp-server -y

# Makes the DHCP service start on boot.
sudo systemctl enable isc-dhcp-server

# Checks the status. It may show inactive at first — that's normal before
config.
sudo systemctl status isc-dhcp-server
```

The service may initially show as inactive until configuration is completed. This is expected.

---

2. Specify the Interface

Edit the DHCP defaults file:

```
# Opens the config that tells DHCP which network interface to serve.
sudo nano /etc/default/isc-dhcp-server
```

Set your internal network interface (for VirtualBox usually `enp0s8`):

```
INTERFACESv4="enp0s8"
INTERFACESv6=""
```

Save and exit.

3. Create the Main Configuration File

Backup the original configuration and edit:

```
# Creates a backup of the default config.
sudo cp /etc/dhcp/dhcpd.conf /etc/dhcp/dhcpd.conf.original

# Opens the main DHCP config file to edit.
sudo nano /etc/dhcp/dhcpd.conf
```

Replace contents with:

```
# DHCP configuration for student22503.lan

option domain-name "student22503.lan";
# Sets the domain name given to clients.

option domain-name-servers 192.168.56.10;
# Tells clients to use your DNS server.

default-lease-time 86400;
# Lease lasts 24 hours by default.

max-lease-time 86400;
# Maximum lease time also 24 hours (keeps things simple for tests).

ddns-update-style none;
# Disable dynamic DNS updates (common for simple setups).

authoritative;
# This DHCP server is the main and correct one for this network.

subnet 192.168.56.0 netmask 255.255.255.0 {
    range 192.168.56.150 192.168.56.200;
    # Clients will get IPs between 150 and 200.

    option routers 192.168.56.10;
    # Default gateway for clients (your server/router).

    option broadcast-address 192.168.56.255;
    # Standard broadcast address for this subnet.
}
```

Save and exit.

---

4. Validate and Start the Service

```
# Checks the config file for errors before starting the service.
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf

# Restarts the DHCP service with your new config.
sudo systemctl restart isc-dhcp-server

# Should show: active (running).
sudo systemctl status isc-dhcp-server

# Shows if DHCP is listening on UDP port 67 (correct).
sudo ss -ulnp | grep dhcpd
```

Expected: DHCP should be **active (running)** and listening on UDP port **67**.

---

5. Test on the Client

Identify the client's interface (usually `enp0s8`):

```
# Shows the client's interfaces and current IP (before obtaining DHCP).
ip addr show
```

Release and renew the IP lease:

```
# Installs the DHCP client tools — needed for dhclient.
sudo apt install isc-dhcp-client -y

# Releases the current IP address.
sudo dhclient -r enp0s8

# Requests a new IP from your DHCP server (with verbose output).
sudo dhclient -v enp0s8
```

Verify configuration:

```
# Should show an IP between 192.168.56.150–200.
ip addr show enp0s8

# Should show default route via 192.168.56.10.
ip route show

# Should show DNS server: 192.168.56.10 and domain student22503.lan.
cat /etc/resolv.conf
```

Expected:

- IP between **192.168.56.150–200**
- Gateway and DNS: **192.168.56.10**
- Domain: **student22503.lan**

---

6. Check Leases on the Server

```
# Shows all DHCP leases the server has given.
sudo cat /var/lib/dhcp/dhcpd.leases

# Filters for active clients and shows details (MAC, IP, expiry).
grep "binding state active" /var/lib/dhcp/dhcpd.leases -A 3
```

This confirms which clients have active leases.

---

7. Verify Integration with DNS

Your DHCP configuration already distributes your local DNS automatically.

From the client, test:

```
# Local DNS resolution should work.
dig server.student22503.lan

# External resolution works through DNS forwarders.
dig google.com
```

Expected results:

- Local hostname resolves via your DNS.
- External domains resolve via forwarders.

---

8. Verification Checklist

Test	Expected Result
`systemctl status isc-dhcp-server`	Shows "active (running)"
`ip a` (on client)	IP between 192.168.56.150–200
`cat /etc/resolv.conf`	DNS = 192.168.56.10
`dig server.student22503.lan`	Resolves local domain
`dig google.com`	Resolves external domain
`ping 8.8.8.8`	Internet reachable

DHCP is now fully set up

Your DHCP server is successfully handing out IP addresses and network information. Clients receive the correct IP range, gateway, DNS, and domain name. Leases are created properly and the service is stable. With DHCP running, your network is now much easier to scale and maintain.

DHCP Troubleshooting Common Issues

This guide helps you diagnose and fix common issues in your **DHCP server configuration**, including service failures, missing leases, and client-side problems.

---

1. Check if the DHCP Service Is Running

```
sudo systemctl status isc-dhcp-server
```

Expected: `active (running)`

If it's NOT running:

```
sudo systemctl restart isc-dhcp-server
sudo journalctl -u isc-dhcp-server -n 20
```

Look for errors such as:

- Wrong interface
- Invalid config syntax
- Missing lease file

---

2. Validate the DHCP Configuration File

Check for syntax errors:

```
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf
```

Fix issues like:

- Missing semicolons `;`
- Wrong braces `{ }`
- Invalid IP ranges
- Incorrect subnet mask

---

3. Confirm DHCP Is Bound to the Correct Interface

```
cat /etc/default/isc-dhcp-server
```

Expected:

```
INTERFACESv4="enp0s8"
```

If incorrect → DHCP will NOT respond.

---

4. Ensure Server Has Correct IP Address

```
ip addr show enp0s8
```

Expected:

```
192.168.56.10/24
```

If wrong, fix Netplan and apply:

```
sudo netplan apply
```

---

5. Test Client Connectivity to DHCP Server

From client:

```
ping -c 2 192.168.56.10
```

If ping fails:

- Wrong adapter type
- Host-only not connected
- Interface down

---

6. Release & Renew IP on Client

```
sudo dhclient -r enp0s8
sudo dhclient -v enp0s8
```

If you see:
"No DHCPOFFERS received"

This means:

- Server not reachable
- Server not running
- Firewall blocking UDP 67/68
- Wrong interface configured

7. Check DHCP Listening Ports

```
sudo ss -ulnp | grep dhcp
```

Expected:

```
0.0.0.0:67  (udp)
```

If missing → the service is not listening.

---

8. Firewall Blocking DHCP

If UFW is enabled:

```
sudo ufw status
```

Allow DHCP:

```
sudo ufw allow 67/udp
sudo ufw allow 68/udp
sudo ufw reload
```

---

9. Check DHCP Lease File (Server)

```
sudo cat /var/lib/dhcp/dhcpd.leases
```

Look for:

```
binding state active
```

If the file is empty → no clients got an IP.

---

10. Verify Client Routing & DNS

Routing

```
ip route show
```

Expected:

```
default via 192.168.56.10
```

DNS

```
cat /etc/resolv.conf
```

Expected:

```
nameserver 192.168.56.10
```

If missing, the client received *partial* DHCP info.

---

11. Subnet Mask Mismatch

Both server and client MUST use:

```
255.255.255.0
```

Mismatch causes:

- Wrong network detection
- DHCP server unreachable

---

12. Debug DHCP Traffic with tcpdump

On the server:

```
sudo tcpdump -i enp0s8 port 67 or port 68
```

You should see:

- DHCPDISCOVER from client
- DHCPOFFER from server

If only DISCOVER appears → server isn't responding.

---

13. Reset DHCP State

If leases or state files are corrupted:

```
sudo systemctl stop isc-dhcp-server
sudo rm /var/lib/dhcp/dhcpd.leases
sudo touch /var/lib/dhcp/dhcpd.leases
sudo systemctl start isc-dhcp-server
```

---

14. Test Static IP to Confirm Network Works

If DHCP still fails:

```
sudo ip addr add 192.168.56.50/24 dev enp0s8
sudo ip route add default via 192.168.56.10
```

If internet works with static IP → the DHCP config is the only problem.

FTP Server Configuration

This section covers building a secure FTP server using vsftpd. You will allow authenticated access, upload/download support, and passive mode for better compatibility. By the end, you will have a secure and functional FTP server that accepts valid login credentials and supports file transfers.

---

1. Install and Enable vsftpd

```
# Shows if vsftpd is running. It should say "active (running)".
sudo apt update
```

```
# Installs the vsftpd FTP server.
sudo apt install vsftpd -y

# Makes vsftpd start automatically on boot.
sudo systemctl enable vsftpd

# Starts the FTP server right now.
sudo systemctl start vsftpd

# Shows if vsftpd is running. It should say "active (running)".
sudo systemctl status vsftpd
```

Confirm that the service shows `active (running)` before continuing.

---

2. Configure vsftpd

Backup and edit the configuration file:

```
# Creates a backup of the original config—useful if something breaks later.
sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.backup

# Opens the main vsftpd configuration file.
sudo nano /etc/vsftpd.conf
```

Replace or ensure the following lines are present and **uncommented**:

```
# ================================
# Authenticated access only
# ================================

listen=YES
# FTP listens on IPv4.

listen_ipv6=NO
# Disable IPv6 listening (keeps setup simple).

anonymous_enable=NO
# Disable anonymous FTP for security.

local_enable=YES
# Allow real Linux users to log in.

write_enable=YES
```

```
# Allow upload, rename, and delete actions.

chroot_local_user=YES
# Lock users in their home directory for safety.

allow_writeable_chroot=YES
# Needed so users can write while chrooted.

pasv_enable=YES
# Enable passive mode (needed behind NAT or firewalls).

pasv_min_port=40000
pasv_max_port=40100
# Passive mode port range—keeps firewall rules simple.

xferlog_enable=YES
# Log all uploads/downloads.

xferlog_std_format=NO
# Use modern logging format.

xferlog_file=/var/log/vsftpd.log
# Where logs are saved.

dirmessage_enable=YES
# Show messages from .message files when entering directories.

use_localtime=YES
# Show times based on server's local timezone.

connect_from_port_20=YES
# Data connections come from port 20 (standard FTP behavior).

secure_chroot_dir=/var/run/vsftpd/empty
# Safe directory for vsftpd to use when chrooting.

pam_service_name=vsftpd
# Use the correct PAM authentication profile.

ssl_enable=NO
# SSL/TLS disabled (simple setup).
```

Save and close (`Ctrl + O` , `Ctrl + X`).

---

3. Configure Firewall

```
# Installs UFW firewall if not installed.
sudo apt install ufw -y

# Enables UFW (you may be prompted to continue).
sudo ufw enable
```

Open required FTP ports and reload the firewall:

```
# Opens FTP command port.
sudo ufw allow 21/tcp

# Opens passive mode data transfer ports.
sudo ufw allow 40000:40100/tcp

# Reloads firewall rules to apply changes.
sudo ufw reload

# Shows the active firewall rules.
sudo ufw status
```

Verify that both rules appear in the output.

---

4. Restart vsftpd

```
# Restart the service to apply new configuration.
sudo systemctl restart vsftpd

# Confirm it’s running with no errors.
sudo systemctl status vsftpd
```

Check that the status shows `active (running)` and verify it is listening:

```
# Checks if vsftpd is listening on port 21.
sudo ss -tuln | grep 21
```

Expected output:

```
LISTEN 0 32 0.0.0.0:21
```

---

5. Create FTP User

```
# Creates a new Linux user for FTP access.
sudo adduser ftpuser

# Makes a home directory for the FTP user (if not created automatically).
sudo mkdir -p /home/ftpuser

# Gives ownership of the home directory to the FTP user.
sudo chown ftpuser:ftpuser /home/ftpuser

# Sets permissions — readable but not writable by others.
sudo chmod 755 /home/ftpuser
```

This user will be used for authenticated FTP access.

---

6. Test Authenticated FTP Access

From the **client VM (192.168.56.20)**:

```
# Connects to the FTP server.
ftp 192.168.56.10
```

When prompted:

```
Name (192.168.56.10:user): ftpuser
Password: (enter password)
```

Test commands:

```
ftp> pwd Shows current directory on server.
ftp> ls Lists files/directories.
ftp> put test.txt Uploads a file to the server.
ftp> get test.txt Downloads the file back.
ftp> bye Exits FTP session.
```

Expected results:

- Login works only with valid credentials.
- Upload (`put`) and download (`get`) commands succeed.
- Anonymous login is denied.

---

7. Verify Passive Mode

```
ftp 192.168.56.10
ftp> passive # Enables passive mode inside FTP client.
ftp> ls # Tests passive data connection.
```

If you see:

```
229 Entering Extended Passive Mode (|||40xxx|)
150 Here comes the directory listing.
226 Directory send OK.
```

then passive mode is working correctly.

---

8. Logs and Troubleshooting

Check FTP activity logs:

```
# Shows real-time FTP upload/download logs.
sudo tail -f /var/log/vsftpd.log
```

Useful checks:

```
# Shows service errors.
sudo systemctl status vsftpd

# Shows which FTP ports are open.
sudo ss -tuln | grep ftp

# Shows last 20 log entries from the vsftpd service.
sudo journalctl -u vsftpd -n 20
```

9. Verification Checklist

Test	Expected Result
`systemctl status vsftpd`	Shows “active (running)”
Anonymous login	Denied
Authenticated login	Works with `ftpuser`
Upload (put)	Successful
Download (get)	Successful
Passive mode	Functional (ports 40000–40100)
`/var/log/vsftpd.log`	Shows client connection and transfer entries

FTP is now fully set up

Your FTP server is fully configured with secure authentication, working passive mode, and proper firewall rules. The FTP user can upload and download files, and anonymous access is blocked. Logs are available for monitoring, and the server is stable across reboots. You now have a safe file-transfer service on your network.

FTP (vsftpd) Troubleshooting Common Issues

This guide helps you diagnose and fix common issues with your **FTP server**, including login failures, passive mode problems, firewall issues, and file permission errors.

---

1. Check if vsftpd Service Is Running

```
sudo systemctl status vsftpd
```

Expected: `active (running)`

If not:

```
sudo systemctl restart vsftpd
sudo journalctl -u vsftpd -n 20
```

Look for errors such as:

- Syntax issues in vsftpd.conf
- Port conflicts
- Permission errors

---

2. Validate the vsftpd Configuration File

Check the file for mistakes:

```
sudo nano /etc/vsftpd.conf
```

Common issues:

- `anonymous_enable=YES` left enabled
- Missing passive port range
- Bad chroot settings
- IPv6 mistakenly enabled (`listen_ipv6=YES`)

Restart after changes:

```
sudo systemctl restart vsftpd
```

---

3. Check That FTP Ports Are Open

vsftpd needs:

- **Port 21** (FTP command)
- **Passive range: 40000--40100**

Check ports:

```
sudo ss -tuln | grep -E '21|40000'
```

If nothing appears → firewall or config issue.

---

4. Check Firewall Rules

If using UFW:

```
sudo ufw status
```

Allow FTP:

```
sudo ufw allow 21/tcp
sudo ufw allow 40000:40100/tcp
sudo ufw reload
```

Without these, passive mode won't work.

---

5. Verify FTP User Exists

```
sudo getent passwd ftpuser
```

If missing → recreate:

```
sudo adduser ftpuser
```

---

6. Check Permissions on Home Directory

vsftpd requires correct ownership:

```
ls -ld /home/ftpuser
```

Expected:

```
drwxr-xr-x ftpuser ftpuser /home/ftpuser
```

Fix:

```
sudo chown ftpuser:ftpuser /home/ftpuser
sudo chmod 755 /home/ftpuser
```

---

7. Fix "500 OOPS: vsftpd: refusing to run with writable root"

If you see this error:

Edit config:

```
allow_writeable_chroot=YES
```

Restart:

```
sudo systemctl restart vsftpd
```

---

8. Debug Login Issues

Try logging in from the client:

```
ftp 192.168.56.10
```

Failures may mean:

- Wrong username/password
- User shell disabled
- Incorrect PAM settings

Check authentication logs:

```
sudo journalctl -u vsftpd -n 20
```

---

9. Check Passive Mode Functionality

From the FTP client:

```
ftp> passive
ftp> ls
```

If it hangs:

- Passive ports not open
- Wrong NAT config
- Firewall blocking connections

Ensure vsftpd.conf includes:

```
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=40100
```

---

10. Test Upload/Download Issues

If upload fails:

```
write_enable=YES
```

Check filesystem permissions:

```
sudo chmod 755 /home/ftpuser
sudo chown ftpuser:ftpuser /home/ftpuser
```

---

11. View FTP Server Logs

Real-time logs:

```
sudo tail -f /var/log/vsftpd.log
```

System logs:

```
sudo journalctl -u vsftpd -n 50
```

Look for:

- Permission denied
- Failed login
- Passive connection blocked

---

12. Reset FTP Configuration if Broken

```
sudo mv /etc/vsftpd.conf /etc/vsftpd.conf.broken
sudo cp /etc/vsftpd.conf.backup /etc/vsftpd.conf
sudo systemctl restart vsftpd
```

NFS Server Configuration

This section teaches you how to set up an NFS file server to share folders between machines. You will configure shared directories, read/write and read-only access, SGID group collaboration, firewall rules, and client mounts. By the end, your client VM will be able to mount and use shared directories seamlessly.

---

1. Install and Enable the NFS Server

```
# Updates package list so you get the newest software.
sudo apt update

# Installs the NFS server package.
sudo apt install nfs-kernel-server -y

# Makes NFS start automatically at boot.
sudo systemctl enable nfs-kernel-server

# Starts the NFS server right now.
sudo systemctl start nfs-kernel-server

# Shows if NFS is running. Should say "active (running)".
sudo systemctl status nfs-kernel-server
```

Confirm that the service is **active (running)**.

2. Create Shared Directories

```
# Folder for read/write sharing.
sudo mkdir -p /home/denis/shared

# Folder for read-only access.
sudo mkdir -p /home/denis/readonly

# Creates a test file inside the shared folder.
echo "NFS Shared Directory Content" | sudo tee /home/denis/shared/test.txt

# Creates a file in the read-only folder.
echo "NFS Readonly Content" | sudo tee /home/denis/readonly/info.txt

# Sets safe permissions that allow reading but controlled writing.
sudo chmod 755 /home/denis/shared
sudo chmod 755 /home/denis/readonly
```

3. Configure Group Collaboration (SGID)

```
# Creates a group for people who can collaborate on the shared folder.
sudo groupadd denisnfs

# Adds your user to the denisnfs group.
sudo usermod -aG denisnfs $USER

# Sets the folder's group to denisnfs.
sudo chgrp denisnfs /home/denis/shared

# Makes your user the owner, and denisnfs the group.
sudo chown -R denis:denisnfs /home/denis/shared

# The 2 sets the SGID bit so new files inherit the folder's group.
sudo chmod 2775 /home/denis/shared

# Should show: drwxrwsr-x - "s" means SGID is active.
ls -ld /home/denis/shared
```

Expected output should include `drwxrwsr-x`, showing the **SGID bit** is set.

4. Configure Exports

Edit the NFS exports file:

```
# Opens the exports file — defines which folders are shared.
sudo nano /etc/exports
```

Add the following lines:

```
# NFS Exports

# Shared folder, read/write access for your whole network.
# rw  → Clients can read and write
# sync → Writes are committed to disk immediately (safer)
# no_subtree_check → Avoids subtree verification, improves reliability
/home/denis/shared   192.168.56.0/24(rw,sync,no_subtree_check)

# /home/denis/readonly settings:
# ro  → Read-only (clients cannot modify files)
# sync → Safer write behavior
# no_subtree_check → Avoid subtree scanning issues
/home/denis/readonly 192.168.56.0/24(ro,sync,no_subtree_check)
```

Apply and verify configuration:

```
# Applies all NFS exports.
sudo exportfs -a

# Shows detailed export info.
sudo exportfs -v

# Shows the list of exported folders.
sudo showmount -e localhost

# Restarts NFS service to apply changes.
sudo systemctl restart nfs-kernel-server
```

---

6. Verify Services and RPC

```
# Checks if NFS is listening on port 2049 (required for NFSv4).
sudo ss -tuln | grep 2049

# Lists RPC services, including nfs, mountd, and other required daemons.
sudo rpcinfo -p localhost
```

Ensure that **port 2049** and **mountd (32767)** appear in the output.

---

7. Configure Firewall

Allow access for your subnet:

```
# Allows NFS service from your local network.
sudo ufw allow from 192.168.56.0/24 to any port 2049

# Allows portmap/rpcbind over TCP and UDP.
sudo ufw allow from 192.168.56.0/24 to any port 111 proto tcp
sudo ufw allow from 192.168.56.0/24 to any port 111 proto udp

# Allows mountd on its fixed port.
sudo ufw allow from 192.168.56.0/24 to any port 32767

# Reload and verify the firewall rules.
sudo ufw reload
sudo ufw status
```

---

8. Client Setup (192.168.56.20)

Install the NFS client:

```
# Installs the client-side NFS utilities.
sudo apt update
sudo apt install nfs-common -y
```

Create mount points:

```
# Folders where the NFS shares will be mounted.
sudo mkdir -p /mnt/denis-shared
```

```
sudo mkdir -p /mnt/denis-readonly
```

Check server exports:

```
# Shows NFS services available on the server.
rpcinfo -p 192.168.56.10

# Shows exported filesystems
showmount -e 192.168.56.10 # if this hangs/fails scroll down and follow
steps 8.1-8.3, if not you can skip them.
```

Expected output:

```
Export list for 192.168.56.10:
/home/denis/readonly 192.168.56.0/24
/home/denis/shared   192.168.56.0/24
```

8.1. Fix for Hanging `showmount -e` (Firewall and Mountd Port)

If `showmount -e 192.168.56.10` hangs on the client, follow these steps:

8.2 Lock the mountd port on the server

```
# Opens NFS config to set static port.
sudo nano /etc/nfs.conf
```

Under `[mountd]`, add:

```
[mountd]
port=32767
# Fixes mountd port so the firewall can allow it properly.
```

Then restart:

```
# Applies new NFS and mountd settings.
sudo systemctl restart nfs-kernel-server
sudo systemctl restart rpcbind
sudo exportfs -a
```

8.3 Allow required ports in UFW

```
sudo ufw allow from 192.168.56.0/24 to any port 2049
sudo ufw allow from 192.168.56.0/24 to any port 111 proto tcp
sudo ufw allow from 192.168.56.0/24 to any port 111 proto udp
sudo ufw allow from 192.168.56.0/24 to any port 32767
sudo ufw reload
sudo ufw status
```

After this, `showmount -e 192.168.56.10` should display exports instantly.

---

9. Fix User and Group Permissions

Ensure the client user matches the server's group.

9.1 Check server user groups

On the server:

```
# Shows the user's groups — should include denisnfs.
id denis
```

Should include `1002(denisnfs)`.

9.2 Create same group on client (same GID)

On the client:

```
# Creates group with same GID as server.
sudo groupadd -g 1002 denisnfs

# Adds local user to denisnfs group.
sudo usermod -aG denisnfs denis

# Refresh group membership in current shell.
newgrp denisnfs

# Confirms denisnfs appears in the group list.
id denis
```

Now `denis` should also belong to `denisnfs`.

---

10. Mount and Test

Mount the shares manually:

```
# Mounts NFS shares for testing.
sudo mount -t nfs 192.168.56.10:/home/denis/shared /mnt/denis-shared
sudo mount -t nfs 192.168.56.10:/home/denis/readonly /mnt/denis-readonly
```

Verify mounts:

```
# Confirms the shares are mounted.
mount | grep nfs
df -h | grep mnt
```

Test read/write access:

```
# Read shared file.
cat /mnt/denis-shared/test.txt

# Read readonly file.
cat /mnt/denis-readonly/info.txt

# Write allowed.
echo "Client write test" | tee /mnt/denis-shared/client-test.txt

# Write denied — expected.
echo "This should fail" | tee /mnt/denis-readonly/fail.txt
```

Expected:

- Write succeeds in `/mnt/denis-shared`
- Write fails in `/mnt/denis-readonly` with "Read-only file system"

---

11. Persistent Mounts

Unmount manual mounts:

```
# Unmount before adding to fstab.
sudo umount /mnt/denis-shared
```

```
sudo umount /mnt/denis-readonly
```

Edit fstab for persistent mounts:

```
# Adds permanent NFS mount entries.
sudo nano /etc/fstab
```

Add the following lines:

```
# NFS mounts

192.168.56.10:/home/denis/shared   /mnt/denis-shared   nfs
defaults,_netdev,soft,timeo=10 0 0
# Server export path  →  Local mount point
# defaults    → Standard mount options
# _netdev     → Wait for network before mounting (important for NFS)
# soft        → Fail softly if server doesn't respond (prevents hangs)
# timeo=10    → Timeout value for retries
# 0 0         → Do not dump/Do not fsck

192.168.56.10:/home/denis/readonly /mnt/denis-readonly nfs
defaults,_netdev,ro,soft,timeo=10 0 0
# Same as above but read-only (ro)
# Good for shared files that shouldn't be changed

# Ensures mounts load automatically on boot.
```

Reload and verify:

```
# Confirms fstab mounts worked.
sudo mount -a
mount | grep nfs
```

---

12. Verification Checklist

Test	Expected Result
`systemctl status nfs-kernel-server`	Active (running)
`showmount -e 192.168.56.10`	Shows both shares
`/mnt/denis-shared`	Read & Write works

Test	Expected Result
`/mnt/denis-readonly`	Write denied
SGID directory	Group inheritance (`s` bit)
After reboot	Mounts persist
Firewall rules	Open for 111, 2049, and 32767

NFS is now fully set up

Your NFS server now supports reliable file sharing with both read-only and read/write access. The SGID setup allows group collaboration, permissions are correct, firewall rules allow access, and client mounts work—including persistent mounts with /etc/fstab. Your NFS environment is now ready for real-world use.

NFS Troubleshooting Common Issues

This guide helps you diagnose and fix common issues with your **NFS server and client setup**, including mount failures, permission errors, firewall blocking, and SGID problems.

---

1. Check if NFS Service Is Running

```
sudo systemctl status nfs-kernel-server
```

Expected: `active (running)`

If not:

```
sudo systemctl restart nfs-kernel-server
sudo journalctl -u nfs-kernel-server -n 20
```

Look for errors like:

- Bad exports file
- Missing directories

- Port errors

---

2. Validate the Exports File

```
sudo exportfs -v
```

If exports DO NOT appear:

- The exports file may have bad syntax.
- The directory does not exist.

Reapply config:

```
sudo exportfs -a
```

---

3. Check `/etc/exports` for Syntax Errors

Common mistakes:

- Missing space after directory
- Wrong network range
- Wrong option format

Correct examples:

```
/home/denis/shared   192.168.56.0/24(rw,sync,no_subtree_check)
/home/denis/readonly 192.168.56.0/24(ro,sync,no_subtree_check)
```

Restart:

```
sudo systemctl restart nfs-kernel-server
```

---

4. Verify NFS Ports Are Open

NFS v4 requires:

- **2049** (main NFS service)
- **111** (rpcbind)
- **32767** (mountd, if fixed)

Check listening ports:

```
sudo ss -tuln | grep -E '2049|111|32767'
```

If missing → rpcbind or NFS failed to start.

---

5. Check RPC Services

```
sudo rpcinfo -p localhost
```

You should see:

- `nfs`
- `mountd`
- `portmapper` / `rpcbind`

If missing:

```
sudo systemctl restart rpcbind
sudo systemctl restart nfs-kernel-server
```

---

6. Firewall Blocking NFS

If UFW is enabled:

```
sudo ufw status
```

Allow all required ports:

```
sudo ufw allow from 192.168.56.0/24 to any port 2049
sudo ufw allow from 192.168.56.0/24 to any port 111 proto tcp
sudo ufw allow from 192.168.56.0/24 to any port 111 proto udp
```

```
sudo ufw allow from 192.168.56.0/24 to any port 32767
sudo ufw reload
```

---

7. Fix showmount Hanging

If this command hangs on the client:

```
showmount -e 192.168.56.10
```

The issue is usually firewall or mountd port not fixed.

Fix:

Edit:

```
sudo nano /etc/nfs.conf
```

Add:

```
[mountd]
port=32767
```

Restart:

```
sudo systemctl restart rpcbind
sudo systemctl restart nfs-kernel-server
sudo exportfs -a
```

---

8. Client Cannot Mount Shares

Try mounting manually:

```
sudo mount -t nfs 192.168.56.10:/home/denis/shared /mnt/denis-shared
```

If error:

Connection refused

- NFS service not running
- Firewall is blocking

Permission denied

- Wrong export options
- Client IP not in allowed subnet

Stale NFS handle

Fix:

```
sudo umount -f /mnt/denis-shared
sudo exportfs -a
```

---

9. Permission Issues (SGID and Ownership)

Check folder permissions:

```
ls -ld /home/denis/shared
```

Expected:

```
drwxrwsr-x
```

If `s` bit missing:

```
sudo chmod 2775 /home/denis/shared
```

Ensure group matches:

```
sudo chgrp denisnfs /home/denis/shared
```

---

10. Client User Must Match Group

On server:

```
id denis
```

Note the GID of `denisnfs`.

On client:

```
sudo groupadd -g <GID> denisnfs
sudo usermod -aG denisnfs denis
newgrp denisnfs
```

---

11. Client Cannot Write in Shared Directory

Check:

- SGID bit
- Owner and group
- Export options are `rw`

Fix permissions:

```
sudo chown denis:denisnfs /home/denis/shared
sudo chmod 2775 /home/denis/shared
```

---

12. Test File Access

```
touch /mnt/denis-shared/testfile
```

If denied:

- Wrong group
- Client user not in denisnfs
- Permissions incorrect
- Export not `rw`

---

13. Check fstab Mounting Issues

If mount fails on boot:

- `_netdev` missing
- Wrong NFS version

Example entry:

```
192.168.56.10:/home/denis/shared /mnt/denis-shared nfs defaults,_netdev 0 0
```

Then test:

```
sudo mount -a
```

---

14. Debug with tcpdump

On server:

```
sudo tcpdump -i enp0s8 port 2049
```

Look for:

- Client mount attempts
- Packet drops

Final Conclusion

Your server is now running all five required network services — Routing/NAT, DNS, DHCP, FTP, and NFS — at the same time, fully integrated and working together.
The DHCP client can automatically receive an IP address, discover the correct gateway and DNS server, resolve names through your DNS zones, reach the internet through NAT, mount shared folders using NFS, and transfer files securely through FTP.

Every service depends on the others, creating a complete and functional network environment.
DNS supports DHCP. DHCP supports routing. Routing enables internet access.
NFS and FTP both rely on the network configuration and DNS to function properly.
All services start on boot, work reliably after reboots, and include firewall rules and security settings where appropriate.

This setup mirrors how a real-world infrastructure works on private networks.
By completing this configuration, you now have a full understanding of how multiple Linux services can work together to form one stable, interconnected system.